CODESPECT

ETH Staking Deposit Bot

SECURITY ASSESSMENT REPORT

July 7, 2026

Prepared for:

Swell

Contents

1 About CODESPECT

CODESPECT is a specialised smart contract security firm dedicated to ensure the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to build secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialise in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralised applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialised cybersecurity solutions designed to minimise risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implementing a bug bounty programme to incentivise external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

3.1 Impact

- **High** - Results in a substantial loss of assets (more than 10%) within the protocol or causes significant disruption to the majority of users.
- **Medium** - Losses affect less than 10% globally or impact only a portion of users, but are still considered unacceptable.
- **Low** - Losses may be inconvenient but are manageable, typically involving issues like griefing attacks that can be easily resolved or minor inefficiencies such as gas costs.

3.2 Likelihood

- **High** - Very likely to occur, either easy to exploit or difficult but highly incentivised.
- **Medium** - Likely only under certain conditions or moderately incentivised.
- **Low** - Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.3 Action Required for Severity Levels

- **Critical** - Must be addressed immediately if already deployed.
- **High** - Must be resolved before deployment (or urgently if already deployed).
- **Medium** - It is recommended to fix.
- **Low** - Can be fixed if desired but is not crucial.

In addition to High, Medium, and Low severity levels, CODESPECT utilises one other category for findings: **Informational**.

a) **Informational** findings do not pose a direct security risk but provide useful information the audit team wants to communicate formally, including cases where certain portions of the code deviate from established smart contract development standards or best practices.

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for the Swell `eth-staking-deposit-bot` (referred to throughout as the "deposit bot"). The deposit bot is an auditable, step-by-step command-line tool that a Swell operator runs to deposit node-operator validators on Ethereum mainnet, for both the swETH liquid staking token (LST) and the rswETH liquid restaking token (LRT).

The review covered the Python bot only. It is an off-chain operator tooling that holds no protocol funds and has no privileged on-chain role beyond what the operator's signing key is authorised to do; the Swell smart contracts it interacts with (`NodeOperatorRegistry`, `DepositManager`, `EigenLayerManager`, and related contracts) were out of scope. All reviewed files are summarised in subsection 5.1.

The audit was performed using:

a) Manual analysis of the codebase.

b) Dynamic analysis and execution testing of the tool against recorded and forked mainnet state.

Organisation of the document is as follows:

- **Section 5** summarises the audit.
- **Section 6** describes the functionality and architecture of the in-scope tool.
- **Section 7** presents the issues.
- **Section 8** presents the threat model.
- **Section 9** discusses the documentation provided by the client for this audit.
- **Section 10** presents the compilation and tests.

CODESPECT found twelve points of attention, one classified as `High`, two classified as `Medium`, five classified as `Low`, and four classified as `Informational`. All of the issues are summarised in Table 2.

Issues found:

Severity	Unresolved	Fixed	Acknowledged
Critical	0	0	0
High	0	1	0
Medium	0	1	1
Low	0	5	0
Informational	0	3	1
Total	**0**	**10**	**2**

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

Audit Type	Security Review
Project Name	ETH Staking Deposit Bot
Type of Project	Off-chain Operator Tooling (Validator Deposit CLI)
Duration of Engagement	4 Days
Duration of Fix Review Phase	1 Day
Draft Report	July 3, 2026
Final Report	July 3, 2026
Repository	eth-staking-deposit-bot
Commit (Audit)	9298016aca651fef821aad0b9457dd9f4bdb8689
Commit (Final)	ede12615c831e707413e020ec4dfb340f8637aab
Documentation Assessment	High
Test Suite Assessment	Medium
Auditors	JecikPo, WarlordSam07

Table 3: Summary of the Audit

5.1 Scope - Audited Files

	File	LoC
1	src/deposit_bot/plan.py	263
2	src/deposit_bot/web3_chain.py	244
3	src/deposit_bot/cli.py	187
4	src/deposit_bot/deposit_seam.py	183
5	src/deposit_bot/gates.py	137
6	src/deposit_bot/broadcast.py	119
7	src/deposit_bot/bls.py	95
8	src/deposit_bot/key_index.py	95
9	src/deposit_bot/config.py	94
10	src/deposit_bot/web3_broadcast.py	86
11	src/deposit_bot/allocate.py	78
12	src/deposit_bot/witness.py	60
13	src/deposit_bot/chain.py	49
14	src/deposit_bot/env.py	27
15	src/deposit_bot/ids.py	24
16	src/deposit_bot/__init__.py	6
	Total	**1747**

5.2 Findings Overview

	Finding	Severity	Update
1	Deposit front-run of an aged key captures 32 ETH to an operator credential	High	Fixed
2	A single BLS-invalid front-of-queue key can halt allocated node-operator deposits	Medium	Acknowledged
3	Nonce read at `latest` not `pending` causes nonce collision on an in-flight transaction	Medium	Fixed
4	Duplicate operator name silently shadows an operator	Low	Fixed
5	No gate checks the operator `enabled` flag	Low	Fixed
6	Stale or rotated operator address crashes the run with an unhandled traceback	Low	Fixed
7	The `not_on_beacon` gate fails open on an empty, lagging, or hostile beacon feed	Low	Fixed
8	`verify_deposit(...)` crashes instead of returning `False` on malformed-hex input	Low	Fixed
9	Beacon validator query is an un-chunked GET with no error handling and can exceed strict URI limits (HTTP 414)	Info	Fixed
10	`funding_covers_queue` gate never fires on dead code	Info	Fixed
11	Hardcoded fee policy can strand the deposit tx	Info	Fixed
12	Operator can steer redistributed surplus by deepening its own queue	Info	Acknowledged

6 System Overview

The `eth-staking-deposit-bot` (the “deposit bot”) is an auditable, step-by-step command-line tool that a Swell node operator runs to deposit node-operator validators on Ethereum mainnet. It supports two Swell products, **swETH** (the liquid staking token, LST) and **rswETH** (the liquid restaking token, LRT). One run builds and broadcasts exactly one deposit transaction of up to `-cap` validators and then exits; to deposit more, the operator runs it again. It is deliberately a tool a human runs and confirms, not a long-running service: the bot decides *which* keys to deposit and *how many*, but the decision of *when* to run and the final go/no-go on the broadcast are left to the operator.

The tool does two things it is careful to be explicit about **not** doing. It does not generate validator signatures: deposit signatures are registry-stored (operators submitted them on-chain when they registered their keys), and the bot only selects keys and reuses the registry’s signatures. And it does not decide when to deposit: allocation follows the configured target ratios, but the timing is a human decision made outside the bot. The tool’s entire job is to turn "the protocol has surplus ETH and pending keys" into "one correct, verified, operator-confirmed deposit transaction", with a legible audit record of every decision.

The Two Products and Their Withdrawal Credentials

The single meaningful difference between the two modes is each validator’s **withdrawal credential**, the 32-byte value baked into the deposit that decides where the validator’s stake and rewards can later be withdrawn:

- **swETH (LST).** Every validator uses one fixed withdrawal credential: the protocol `DepositManager` address, `0x01`-prefixed. There is nothing to discover; each selected key either verifies against that single credential or it does not. The deposit transaction is `DepositManager.setupValidators(pubKeys, depositDataRoot)`.
- **rswETH (LRT).** rswETH validators restake through EigenLayer, so each validator’s withdrawal credential is its staker’s **EigenPod** (`0x01` followed by the pod address). The bot must therefore determine which pod each key belongs to before it can build the deposit. It does so by **recovery**: it enumerates the on-chain set of staker proxies and their pods, forms each pod’s withdrawal credential, and for each key finds the pod whose credential makes the key’s registry signature verify. A BLS signature commits to exactly one credential, so at most one pod can match. The deposit transaction is `EigenLayerManager.stakeOnEigenLayer(stakerIds, pubKeys, depositDataRoot)`.

Everything else, the allocation, key selection, funding, sequence, pause, and beacon checks, is identical across the two products, because they share the same `NodeOperatorRegistry` semantics and the same queue-seniority invariant.

Architecture

The codebase is organised around these files:

- `plan.py` — the protocol-agnostic planning pipeline. `preview` and `execute` run exactly this same computation up to the send, so "preview shows what execute will do" is a structural property, not a promise.
- `allocate.py` — the stock-based ratio allocation (a pure function).
- `gates.py` — the safety gates, pure predicates that record a verdict rather than raising.
- `deposit_seam.py` — the two protocol-specific pieces: per-key resolution (swETH credential vs rswETH pod recovery) and building the single deposit transaction.
- `bls.py` — deposit-signature verification (verify only, never sign).
- `key_index.py` — the maturity index, a rebuildable SQLite cache of key add-times.
- `witness.py` — the append-only JSONL audit log.
- `chain.py` / `web3_chain.py` — the read seam (interface and web3 implementation).
- `broadcast.py` / `web3_broadcast.py` — the send path (orchestration and web3 signing/sending).
- `cli.py`, `config.py`, `ids.py`, `env.py` — the entry point, dataset loading, content-addressed batch identity, and a minimal `.env` loader.

The Dataset

The bot runs in one mode at a time and loads exactly one **dataset** (a TOML file, `datasets/sweth.toml` or `datasets/rsweth.toml`). A dataset declares:

- the protocol’s contract addresses (`NodeOperatorRegistry`, `DepositManager`, the exit contract that reports `exitingETH`, the `AccessControlManager` that holds the pause flag, and, for rswETH only, the `EigenLayerManager`);

- a per-operator table of `{controlling address, target ratio}` for the live operators. The address is the operator's **controlling** address, the value the `NodeOperatorRegistry` indexes operators by, which is what lets the bot find an operator and enumerate its pending keys;
- `min_key_age`, the maturity window for the key-age gate, and the registry's deploy block, which seeds the maturity index so it need not scan from genesis.

The Workflow of Bot

Both `preview` and `execute` build the same plan from chain; `preview` stops before the broadcast.

a. **Read state.** The key-graph (pending keys, active counts, and for rswETH the EigenPods) is read at the **finalized** block: it cannot reorg, and every pending key there is one the maturity index has already seen, so the plan is one coherent, reproducible snapshot. Money, obligation, and pause (the `DepositManager` balance, the exit contract's `exitingETH`, and the paused flag) are read at **latest**, because they can move against the bot between blocks and the contract re-checks them live.

b. **Size.** `deployable = balance - exitingETH`; the number of fundable validators is `deployable / 32 ETH`, capped to `-cap`. Any surplus beyond the cap is left for the next run.

c. **Allocate.** Distribute the new validators towards the target ratios, placing each new validator with the operator currently furthest below its target share that still has a pending key (a greedy, stock-based allocation).

d. **Select.** Take each operator's allocated count from the **front** of its pending list, in registry order (the exact prefix, no silent truncation).

e. **Resolve and verify.** For each selected key, verify its registry BLS signature and resolve its withdrawal target (for rswETH, recover its EigenPod). There is no unverified path.

f. **Gate.** Run the safety gates; the first refusal blocks the run and is recorded with its reason.

g. **Broadcast (execute only).** Build the single deposit transaction, simulate it, take the operator's confirmation, then sign and send.

7 Findings

7.1 High

7.1.1 [High] Deposit front-run of an aged key captures 32 ETH to an operator credential

File(s): gates.py, web3_chain.py, plan.py

Description: This off-chain bot decides which validator public keys receive a 32 ETH deposit funded from the pooled DepositManager balance. The contracts deliberately deny node operators any control over a validator's withdrawal credential and always substitute the protocol's own (the DepositManager address for swETH, a protocol-controlled EigenPod for rswETH), so on exit the staked ETH returns to the protocol. The setupValidators NatSpec in IDepositManager gives the off-chain service two front-running defenses: validate that each public key has not already been used for a validator setup, and snapshot the deposit contract's depositDataRoot for re-check at broadcast. The bot performs only the snapshot (backed on-chain by InvalidDepositDataRoot) and never the used-key check, so the credential-fixing pre-deposit it is meant to catch goes unchecked. A malicious or compromised operator who holds a validator BLS key:

a. Registers key K with the NodeOperatorRegistry well in advance. The registry stores the pubkey and signature without verifying the signature (its own comment delegates this to an off-chain service, i.e. this bot). The registered signature is valid for the protocol credential, so K later passes the bls_valid gate;
b. Shortly before a run, submits a separate 1 ETH deposit to the beacon deposit contract for K, under a withdrawal credential the operator controls. Consensus fixes a validator's credential on its first valid deposit, so K's credential is now permanently the operator's;
c. The bot later selects K (aged, registered, BLS-valid) and broadcasts the 32 ETH deposit. Consensus treats it as a top-up to an already-initialized validator and credits the 32 ETH to the operator's credential, ignoring the protocol's. Per apply_deposit in the consensus spec, the signature and withdrawal credential are used only for the first deposit to a pubkey; every later deposit only increases the balance;

The gates miss it. key_age_mature measures the time since K was added to the registry (key_added_ts, built from OperatorAddedValidatorDetails logs), not the time since the hostile pre-deposit, so registering K early clears the window with a full margin: gates.py:117-123

```
now = ctx["finalized_ts"]
added = ctx["key_added_ts"]
min_age = ctx["min_key_age_secs"]
# @audit age anchored to registry-add time, not first on-chain deposit; aged key passes despite hostile pre-deposit
too_young = [pk for pk in ctx["pubkeys"] if now - added.get(pk, now) < min_age]
```

not_on_beacon reads the validator set at beacon HEAD, which a freshly pre-deposited key does not enter until the deposit is processed, so the pre-deposit is invisible during that lag; it never reads the deposit contract where the pre-deposit lands immediately. The InvalidDepositDataRoot guard only covers the short read-to-broadcast window, so a pre-deposit that settled earlier does not trigger it. No check asks whether a selected key already received a deposit, and to which credential.

Impact: Each front-run key permanently sends 32 ETH of pooled user funds to a validator whose withdrawal credential the operator controls, recoverable only by the operator on exit. A 1 ETH pre-deposit captures 32 ETH; the loss is direct, irrecoverable, repeatable across keys and runs, and applies to both swETH and rswETH. High if a malicious or compromised node operator is in scope; if operators are fully trusted, the front-run is an accepted risk under that assumption rather than a defect.

Recommendation(s): Before depositing, query the deposit-contract history (or a beacon source exposing prior/pending deposits) for each selected key and refuse the run if one exists under a credential other than the expected protocol or pod credential. Alternatives: anchor key_age_mature to the first on-chain deposit observed for the key rather than the registry-add time, or require operators to self-fund the full 32 ETH so there is no pooled top-up to capture. Separately, make not_on_beacon fail closed by distinguishing an empty beacon response from a complete one and cross-checking beacon finality against the execution finalized block.

Status: Fixed

Update from Swell: Addressed in ede12615. New no_foreign_predeposit gate: before depositing, the bot reads each selected key's existing deposit credentials from the beacon node (Pectra pending_deposits) and halts the whole run if any key already carries a deposit under a credential other than its expected protocol/pod one. The finding's "separately" recommendation — the fail-closed on-beacon read with an EL cross-check — is delivered under CODESPECT-03, and the same checks protect this read. The remaining dependency is the beacon node itself: the bot verifies it is synced, Pectra-aware, and ahead of the EL finalized block, and refuses on any doubt — but a node that lies within those checks is believed. Operator registration is governance-gated, which bounds who could attempt this.

Update from CODESPECT: Fixed (narrow residual). New no_foreign_predeposit gate halts on any key already deposited under a non-protocol credential; still reads at plan time, not re-checked at broadcast.

7.2 Medium

7.2.1 [Medium] A single BLS-invalid front-of-queue key can halt allocated node-operator deposits

File(s): `gates.py`, `plan.py`, `web3_chain.py`

Description: Each run's selected keys pass a fixed sequence of gates before broadcast. `bls_valid` refuses to broadcast if any selected key's registry signature does not verify against its withdrawal credentials, which guarantees the bot never deposits to an unverifiable key. The problem is the scope: the refusal is global over the whole batch, so one invalid key blocks every operator selected in the same run. Three properties combine. First, the registry accepts a bad-signature key with no verification: `NodeOperatorRegistry.addNewValidatorDetails` stores `(pubKey, signature)` without checking the signature (delegated to this bot by design), so any live, enabled operator can register a key whose signature does not commit to the protocol or pod credential, with no special on-chain state required. Second, the bot folds the number of failed keys across all operators into a single count (`bls_unverified = len(resolved.unverified)`, `plan.py:199`) and the gate refuses the whole plan if it is non-zero, with no notion of which operator owns the offending key: `gates.py:126-137`

```
def bls_valid(ctx):
    n_bad = ctx["bls_unverified_count"]
    # @audit batch-global refusal, no per-operator isolation; one bad key halts whole fleet
    if n_bad:
        return f"{n_bad} key(s) failed BLS verification / pod recovery"
    return True
```

Third, selection is FIFO front-of-queue (`selected[o.name] = pending[o.name][:want]`), and the contract consumes each operator's keys strictly front-to-back: `usePubKeysForValidatorSetup` reverts `NextOperatorPubKeyMismatch` unless the next key consumed is the operator's current head, so the bot cannot skip a bad head key. A blocked run deposits nothing and changes no on-chain state, so re-runs against the same snapshot re-select the same head key and fail again. The halt is self-perpetuating until the key is removed on-chain or the selected allocation changes, and under normal target-ratio allocation a live operator is allocated regularly, so this is a realistic operating condition rather than a theoretical edge case.

Impact: A single live node operator, malicious or merely careless with one key, can halt all allocated node-operator deposits protocol-wide (both swETH and rswETH) until a maintainer removes the offending key on-chain. Honest operators selected alongside it are blocked even though their own keys are valid. No funds are lost and no wrong-credential deposit occurs: the gate fires before broadcast, so the 32 ETH per validator stays in the `DepositManager` and nothing is signed. The block is recoverable through a routine on-chain key removal and is legible in the witness log as a clean `deposit.blocked` record at the `bls_valid` gate.

Recommendation(s): Fault-isolate by operator instead of refusing the whole batch. When an operator's front-of-queue key fails `bls_valid`, allocate zero to that operator and proceed with the operators whose front prefixes verify, surfacing the offending operator and key as an incident in the witness log. Isolation must be per-operator (the registry consumes keys front-to-back, so a single bad key inside a prefix cannot be skipped), which stops one operator's bad key from halting the whole fleet while still refusing to deposit any unverifiable key.

Status: Acknowledged

Update from Swell: Addressed in `ede12615`. We considered the per-operator fault isolation and deliberately kept the batch-global halt: an unverifiable key from a vetted operator is an incident, and the right response is to stop and review out-of-band, not to isolate the operator and continue — the same whole-run-stop posture as CODESPECT-02. What we fixed is legibility: the refusal now names the offending operator, key, and cause — in the refusal itself and in the preview — so the review is immediately actionable.

Update from CODESPECT: Acknowledged. One bad key still halts the whole fleet; only the refusal message improved (names the operator, key, and cause)

7.2.2 [Medium] Nonce read at `latest` not `pending` causes nonce collision on an in-flight transaction

File(s): `web3_broadcast.py`

Description: `Web3Broadcaster.prepare` builds the deposit transaction by hand and reads the signing account's transaction count to set the nonce, with no `block_identifier`: `web3_broadcast.py:64`

```
# @audit no block_identifier defaults to "latest", ignores in-flight mempool tx, causing nonce collision
"nonce": self.w3.eth.get_transaction_count(self.account.address),
```

With no `block_identifier`, the call falls back to web3's default block `"latest"`, which returns the count of mined transactions only and ignores transactions already signed and sitting in the mempool. If a previous deposit transaction was submitted but not yet mined, the count at `"latest"` is unchanged, so `get_transaction_count` returns the same nonce the in-flight transaction already occupies. The newly signed transaction takes the colliding nonce, and the node either rejects it (`replacement transaction underpriced`) or silently drops it, leaving the operator to manually cancel the original before a re-run can succeed. web3's own `fill_nonce` reads `"pending"` precisely to include unmined transactions, but the bot hand-builds the transaction dict and never goes through it. The most realistic way to reach the stuck-transaction precondition is the bot's static EIP-1559 fee cap: a base-fee spike above the fixed `maxFeePerGas` strands the first deposit transaction in the mempool, and the operator's manual re-run then signs against the stale `"latest"` count and collides. The same condition arises if the signing key is used concurrently from a second process.

Impact: Denial of service and operational disruption, no fund loss. Dormant under single-run usage where `latest == pending`; it activates when a prior deposit transaction is stuck in the mempool (the condition the static fee cap creates) or when the signing key is used concurrently. In that state the freshly signed transaction collides on nonce with the in-flight one, and the operator must manually cancel the stuck transaction before re-running. No 32 ETH deposit is misdirected and no funds leave the `DepositManager`; the consequence is a blocked run requiring manual intervention.

Recommendation(s): Read the nonce at `"pending"` so the count includes unmined transactions:

```
"nonce": self.w3.eth.get_transaction_count(self.account.address, block_identifier="pending"),
```

Pair this with a fee-bump / replacement path for the deposit transaction; the two deficiencies interact (a stuck transaction plus a stale-nonce re-run produces a collision with no automated recovery), and reading the pending nonce is a prerequisite for any EIP-1559 same-nonce replacement. Add a persistent single-run signer lock keyed by (chain id, signer address, mode) so two runs cannot sign concurrently, and refuse to execute if the witness log holds an unresolved `tx.signed` or `tx.in_flight_uncertain` hash not yet confirmed or reverted on chain.

Status: Fixed

Update from Swell: Addressed in `ede12615`. The nonce is read at `pending`, and the send path refuses to run while the signer has an in-flight transaction (pending > latest) — the stuck-tx re-run path that produced the collision. That check also covers the suggested refusal over the bot's own record of signed transactions: reading the chain catches in-flight transactions signed outside this bot too. We did not add the signer lock: the bot is an interactively-confirmed, human-run CLI, so simultaneous runs from one signer are an operational error, and their collision fails clean on the node. We did not add fee-bump/replacement either: the bot sends one transaction and does not manage the mempool, so a stalled tx is an out-of-band operator step (see CODESPECT-10).

Update from CODESPECT: Fixed. The send path now aborts when `pending > latest`, so a stuck-tx re-run stops instead of colliding on nonce.

7.3 Low

7.3.1 [Low] Duplicate operator name silently shadows an operator

File(s): `config.py`, `plan.py`

Description: The bot allocates deposits by a greedy-deficit ratio scheme, and every allocation and selection dict is keyed by `operator.name`. `load_dataset` (config.py:83-86) applies no name-uniqueness check. Three planning dicts are built by name: `plan.py:116-122`

```
# @audit dicts keyed by o.name; duplicate name silently overwrites, shadowing one operator to zero
active   = {o.name: chain.active_count(o.address, block) for o in live}
pending  = {o.name: chain.pending_keys(o.address, block) for o in live}
capacity = {name: len(keys) for name, keys in pending.items()}
```

When two operators share a name (with distinct addresses), the second overwrites the first in each dict, so the shadowed operator's pending keys, active count, and capacity are replaced by the twin's. The allocator computes slots over the collided maps, and the selection loop iterates all operators but writes `selected[o.name]` (plan.py:177-184), so the shadowed operator deposits zero keys while the surviving twin fills both slots. The batch stays internally consistent (the selection sums to `n`, all keys BLS-valid), so no gate detects the drift and no contract backstop fires. The shortfall witness is itself computed from the same collided dicts, so the off-ratio state is never surfaced. The duplicate-*address* case differs: two distinct names sharing one address select the same keys twice and are caught loudly by the `no_duplicate_pubkeys` gate.

Impact: Config integrity only. One legitimate operator gets zero deposits while its same-name twin is over-funded, so the validator fleet drifts off the protocol's target allocation ratios silently, with no diagnostic surfaced. There is no fund loss and no wrong credential: the 32 ETH still reaches a legitimate operator's valid keys under the correct withdrawal credential. The trigger requires a fully-trusted config author to duplicate a name in the dataset; there is no attacker-reachable path.

Recommendation(s): Assert name uniqueness in `load_dataset` (raise if `len(names) != len(set(names))`), or, more robustly, key all allocation and selection dicts by operator address, which is unique by construction and maps to on-chain operator identity, eliminating the shadowing class of bug at its root.

Status: Fixed

Update from Swell: Addressed in `ede12615`. `load_dataset` now rejects duplicate operator names or addresses (case-normalized) at load. We chose this over re-keying the maps by address: operators are identified by name everywhere the tool reports to a human, so names must be unique anyway — and once that's enforced at load, the by-name maps are safe.

Update from CODESPECT: Fixed. `load_dataset` now rejects duplicate operator names and addresses, so the silent shadowing cannot enter.

7.3.2 [Low] No gate checks the operator `enabled` flag

File(s): `web3_chain.py`, `plan.py`

Description: The on-chain `NodeOperatorRegistry` tracks an `enabled` boolean per operator and enforces it when consuming keys:

```
// NodeOperatorRegistry.usePubKeysForValidatorSetup (L203-205)
if (!getOperatorForOperatorId[operatorId].enabled) revert CannotUseDisabledOperator();
```

The bot fetches the full operator tuple in `active_count` but reads only field index 4 (`activeValidators`) and discards field index 0 (`enabled`): `web3_chain.py:135-142`

```
op = self._nor.functions.getOperatorForOperatorId(op_id).call(...)
# @audit op[0] enabled flag discarded; disabled operator passes all gates, reverts on-chain
return int(op[4])  # activeValidators -- op[0] (enabled) is discarded
```

None of the seven gates reads it. So a key belonging to an operator disabled on-chain between dataset curation and a run (for example, disabled by Swell governance) passes all seven gates: `plan.ok == True`, the preview prints `gates: PASS`, and the operator launches `execute`. The condition surfaces only at the `simulate` step, where `usePubKeysForValidatorSetup` reverts with a bare `CannotUseDisabledOperator`. Because `simulate` runs before the `confirm` prompt, the run aborts before the operator is asked go/no-go, and because the whole batch is one transaction, one disabled-operator key fails the entire batch.

Impact: Liveness / DoS only; no funds move, since `simulate` precedes `broadcast` and the revert is atomic. The operator gets an opaque `CannotUseDisabledOperator` revert rather than a legible gate refusal naming the offending operator, and must manually investigate which operator is disabled before re-running. The trigger is a Swell-side operator disabled on-chain after curation, or a dataset listing an already-disabled operator; there is no attacker-reachable path.

Recommendation(s): Read `op[0]` (already fetched in the same tuple) and add an eighth gate `operators_enabled` that refuses any plan containing a key from a disabled operator, naming the operator. Alternatively, filter disabled operators out of `live_operators()` during chain reads. Either approach replaces the bare simulate revert with a legible, pre-broadcast refusal.

Status: Fixed

Update from Swell: Update from the client: Addressed in `ede12615`. Of the two options we chose the blocking gate (`operators_eligible`), and made it dataset-level: it fires, naming the operator, even if the disabled operator contributed no keys this run — so a stale dataset entry is corrected rather than hidden until the run it happens to be selected in.

Update from CODESPECT: Fixed. New `operators_eligible` gate reads the on-chain `enabled` flag and blocks a disabled operator pre-broadcast instead of an opaque simulate revert.

7.3.3 [Low] Stale or rotated operator address crashes the run with an unhandled traceback

File(s): web3_chain.py (pending_keys), web3_chain.py (active_count), cli.py

Description: The bot reads each operator's on-chain state through two paired calls. active_count resolves the operator id through the public mapping getter and tolerates an unknown address (if op_id == 0: return 0, since the getter never reverts). pending_keys has no such guard: web3_chain.py:144-148

```
# @audit no op_id == 0 guard; reverts NoOperatorFound for a stale address
details = self._nor.functions.getOperatorsPendingValidatorDetails(addr).call(block_identifier=bid)
```

On-chain, getOperatorsPendingValidatorDetails routes through _getOperatorIdSafe, which reverts on the exact op_id == 0 case active_count swallowed:

```
// NodeOperatorRegistry.sol _getOperatorIdSafe
operatorId = getOperatorIdForAddress[_operatorAddress];
if (operatorId == 0) {
    revert NoOperatorFound(_operatorAddress);
}
```

The public mapping returns 0 for any deleted address, and updateOperatorControllingAddress deletes the old controlling address. So if an operator rotates their controlling wallet, or the dataset lists a typo'd, not-yet-registered, or since-removed address, active_count(addr) (plan.py:116) returns 0 gracefully but pending_keys(addr) (plan.py:121) reverts NoOperatorFound. web3 surfaces this as a ContractLogicError, which is not a BroadcastError; the build_plan call at cli.py:120 is unwrapped, and the only handler (cli.py:147) catches BroadcastError alone, so the process dies with a raw traceback before any gate runs and with no *.blocked witness record. This differs from the disabled-operator case, which reaches a clean simulate revert and a witness record after the gates; here the crash happens during planning, before any gate.

Impact: Whole-run liveness DoS with the worst failure shape in the codebase: an unhandled traceback rather than a structured abort, and no witness record. A single stale or typo'd operator row takes down the entire run. No funds move; the 32 ETH per validator stays in the DepositManager. The trigger is a fully-trusted config author's stale/typo'd dataset address or an on-chain rotation (updateOperatorControllingAddress) after curation; there is no attacker-reachable path. Applies to both products.

Recommendation(s): Make pending_keys mirror active_count: resolve the operator id first and return [] when op_id == 0, so a misconfigured or rotated operator simply contributes no keys and the existing shortfall and witness machinery records it legibly. Alternatively, pre-validate at load that every dataset operator address resolves to a nonzero on-chain operator id, turning the mid-planning crash into a legible refusal.

Status: Fixed

Update from Swell: Addressed in ede12615. pending_keys now mirrors active_count — resolving the operator id first and returning empty for an unresolved address — and the operator is then blocked, named, by the same operators_eligible gate as CODESPECT-04.

Update from CODESPECT: Fixed. A rotated or unresolved address now yields a clean gate block instead of a raw NoOperatorFound traceback.

7.3.4 [Low] The not_on_beacon gate fails open on an empty, lagging, or hostile beacon feed

File(s): web3_chain.py, gates.py

Description: Gate 5 (not_on_beacon) is the bot's primary check that a selected key has not already received a beacon-chain deposit. It is a plain set intersection of the selected pubkeys against a set built from the beacon REST response, where data.get("data", []) treats a missing array as empty: web3_chain.py:160-162

```
with urllib.request.urlopen(req, timeout=30) as resp:
    data = json.load(resp)
# @audit missing/empty data array silently yields empty set; gate fails open
return {v["validator"]["pubkey"].lower() for v in data.get("data", [])}
```

A key absent from that set is treated as not-on-beacon, with no confirmation that the node is synced, canonical, serving a complete response, or even reachable, and the bot uses a single –beacon URL with no fallback or health check. So an empty body ("data":[]), a partial or MITM-injected response, or an HTTP 200 from a minority fork makes beacon_known empty or incomplete, and any selected key absent from it passes regardless of true on-chain state: absence of evidence is treated as evidence of absence. An empty –beacon "" value also passes argparse's required=True while being falsy, silently disabling the gate, since beacon_known returns an empty set whenever beacon_url is falsy.

Impact: Standalone, a defense-in-depth weakening (liveness; the degraded-feed case needs no attacker). Its material risk is as an amplifier for the deposit front-run of an aged key: that attack relies on a window during which the attacker's pre-deposit is not yet visible on the beacon validator set, and an unreliable or operator-controlled feed can extend that window across multiple runs. Low standalone.

Recommendation(s): Absence cannot be the integrity signal, because on a normal batch every selected key is legitimately not yet on beacon (data == []), so a naive "refuse if data == []" would block every deposit and still fail open on a partial response. Use out-of-band positive confirmation instead:

- Include a sentinel id=<known-active-validator> in the same query (the URL builder already joins arbitrary id= params) and fail closed if it is absent from data[];
- Query /eth/v1/node/syncing and refuse unless is_syncing == false;
- Cross-check the beacon head slot against the execution-layer finalized block;
- Consider two independent beacon endpoints and block the run if they disagree;

Also reject an empty or whitespace-only –beacon value at startup.

Status: Fixed

Update from Swell: Update from the client: Addressed in ede12615. Implemented as recommended: an empty result is trusted only when the node reports synced, its head slot (converted to wall-clock time) is ahead of the EL finalized block, and a configured sentinel validator appears in the response; a transport error, a syncing node, a stale head, or a missing sentinel refuses. A beacon endpoint is now mandatory and an empty –beacon is rejected. We did not adopt the remaining "consider" item (two independent beacon endpoints) — a second trusted endpoint adds operational surface for little marginal benefit once freshness is cross-checked.

Update from CODESPECT: Fixed. not_on_beacon now fails closed unless the feed proves itself (synced, sentinel present, head ahead of EL finality); empty –beacon rejected.

7.3.5 [Low] `verify_deposit(...)` crashes instead of returning `False` on malformed-hex input

File(s): `bls.py`

Description: `verify_deposit(...)` is the bot's signature check on data from an untrusted RPC, and its docstring promises a fail-closed contract: malformed input returns `False`, never raises. It honours that for a wrong byte length (the `if len(...)` guard) and a malformed curve point (caught by `try/except ValueError`), but not for malformed hex, because the hex decode runs before the `try`:

```
pk, wc, sig = _b(pubkey), _b(withdrawal_credentials), _b(signature)  # outside the try
if len(pk) != 48 or len(wc) != 32 or len(sig) != 96:
    return False
# ...
try:
    return bool(PopSchemeMPL.verify(...))
except ValueError:
    return False
```

`_b(...)` calls `bytes.fromhex(...)`, which raises `ValueError` on an odd-length or non-hex string, so such input escapes `verify_deposit(...)` and crashes the run instead of returning `False`. This is fail-safe (a crash never returns a wrong `True`) and is not reachable through the normal pipeline (web3's `_hex(...)` always emits even-length `0x...`); the issue is the violation of the documented "never raises" contract.

Impact: Fail-closed-by-crash: no wrong verification and no fund risk, but malformed input aborts the run with an uncaught exception instead of the documented `False`.

Recommendation(s): Move the `_b(...)` parsing inside the `try` (or wrap parse, length-check, and verify in one `try/except ValueError`) so every malformed input returns `False`.

Status: Fixed

Update from Swell: Addressed in `ede12615`. The hex parse moved inside the `try`, so malformed input returns `False` per the documented contract.

Update from CODESPECT: Fixed. The hex parse moved inside `try/except`, so malformed input returns `False` instead of crashing the run.

7.4 Info

7.4.1 [Info] Beacon validator query is an un-chunked GET with no error handling and can exceed strict URI limits (HTTP 414)

File(s): web3_chain.py

Description: beacon_known asks the beacon node which selected pubkeys are already known on the beacon chain (feeding the not_on_beacon gate). It concatenates every selected pubkey into one GET query string and issues a single un-chunked request with no error handling: web3_chain.py:154-161

```
# @audit all pubkeys in one un-chunked GET, no try/except around the call; a transport error aborts the whole run
q = "&".join(f"id={urllib.parse.quote(pk)}" for pk in pubkeys)
url = f"{self.beacon_url.rstrip('/')}/eth/v1/beacon/states/head/validators?{q}"
```

Two issues. First, there is no try/except around the request, so any transport failure (a 414, a 5xx, a dropped connection) propagates out of build_plan and aborts the whole run, taking the not_on_beacon gate down with it. Second, the query length scales with the batch: each pubkey is 98 characters and contributes id= plus the key, so at the default -cap 64 the request URL is about 6.6 KB. Common beacon-fronting proxies accept that at their defaults (nginx caps the request line at 8 KB; Apache's LimitRequestLine is 8190 B), so it does not fail on a typical deployment. It returns HTTP 414 only on a stricter endpoint (for example an IIS-style front with maxUrl 4096 / maxQueryString 2048) or once the cap is raised past roughly 78 keys, where the URL crosses 8 KB.

Impact: Availability and robustness only; no fund loss and no wrong deposit. On the common proxy defaults the default-cap batch is accepted, so this is a hardening item rather than a guaranteed failure. Where it does fire, the cause (URL length, not a beacon-data problem) is not obvious from the raised transport error, and the not_on_beacon gate cannot run for that batch. It is not attacker-triggerable and applies to both products, since the beacon read is shared.

Recommendation(s): Use the beacon API's POST /eth/v1/beacon/states/state_id/validators form, which carries the validator IDs in the request body and is the spec's intended mechanism for large ID lists, avoiding URI limits entirely. Alternatively, chunk the GET into batches of about 20 keys and union the responses. In either case, wrap the call so a transport failure fails the gate closed rather than aborting the whole run.

Status: Fixed

Update from Swell: Addressed in ede12615. The validator query now uses the beacon API's POST form (ids in the body). The error-handling half of this finding is covered by the CODESPECT-03 fix, which fails the beacon read closed on any transport error.

Update from CODESPECT: Fixed. Beacon query is now a POST with ids in the body and wrapped to fail closed, so no HTTP 414 and no run-aborting transport error.

7.4.2 [Info] funding_covers_queue gate never fires on dead code

File(s): gates.py

Description: funding_covers_queue(...) is one of the seven safety gates. Its stated job is to be an early, legible mirror of the DepositManager's on-chain invariant, the contract reverts with InsufficientETHBalance unless the manager holds enough ETH to cover the new validators plus the ETH already owed to exiting validators. The gate is meant to catch that underfunding before any gas is spent. In the funding_covers_queue:

```
need = ctx["n"] * (32 * 10**18) + ctx["exiting_eth_wei"]
have = ctx["deposit_manager_balance_wei"]
if have < need:
    return (
        f"underfunded: balance {have} < {ctx['n']}*32e18 + exitingETH "
        f"{ctx['exiting_eth_wei']} = {need}"
    )
```

the if condition will never be hit, because n is taken from the have and the existingETH, using the same 32-ETH constant, they are taken from the same source. Looks like nothing in the test suite tries to fire that gate, the only one aimed at underfunding is when balance == exiting

Impact: Dead code. Gate does not capture the condition, yet it is already enforced by the correct calculation.

Recommendation: Remove the gate or reconsider when the gate would still make sense (e.g. if the values are re-read)

Status: Fixed

Update from Swell: Addressed in ede12615. Removed the gate and relocated the invariant it was reaching for ("sizing never overspends the balance") into a unit test over a pure sizing function. We did not take the re-read alternative: a fresh-read funding check already exists in simulate() (InsufficientETHBalance) just before broadcast, so a re-armed gate would duplicate it.

Update from CODESPECT: Fixed. The tautological funding gate is gone; the invariant is now a tested pure function and simulate() still re-checks the balance at broadcast.

7.4.3 [Info] Hardcoded fee policy can strand the deposit tx

File(s): web3_broadcast.py

Description: prepare(...) builds and signs the run's single deposit transaction and sets its EIP-1559 fees with a fixed policy and no bump/replace path:

```
base = self.w3.eth.gas_price
txn["maxFeePerGas"] = base * 2
txn["maxPriorityFeePerGas"] = min(base, Web3.to_wei(2, "gwei"))
```

Despite the name, base is eth.gas_price (a suggested all-in gas price, not the EIP-1559 base fee), so maxFeePerGas is only 2x the current suggestion and the tip is capped at 2 gwei. These are read at prepare(...) time, after the human confirm, so the exposure is the prepare-to-inclusion window: if the base fee climbs past the 2x headroom (it can roughly double in 6 blocks under sustained congestion) or a flat 2 gwei tip is uncompetitive, the tx is underpriced and stalls unmined with no automatic bump. A stalled tx then feeds the in-flight re-run path above.

Impact: Liveness only; no fund loss, but the deposit transaction can stall in the mempool with no way to bump or replace it.

Recommendation(s): Make the fee policy configurable (or use a larger/dynamic headroom and a competitive tip), and add a bump-and-replace path for a stuck tx.

Status: Fixed

Update from Swell: Addressed in ede12615. Fees are now dynamic EIP-1559 read at send time — maxFeePerGas = baseFee × headroom + node-suggested tip, both configurable via CLI/env, with intentionally no hard ceiling (a ceiling would re-introduce exactly the stranding this finding describes). We did not add bump-and-replace: the bot sends one transaction and does not manage the mempool (consistent with CODESPECT-01), so a stalled tx is an out-of-band operator action.

Update from CODESPECT: Fixed. Dynamic EIP-1559 (real baseFee, configurable headroom and tip, no hard cap) replaces the old base*2 / 2 gwei-cap that stranded the tx.

7.4.4 [Info] Operator can steer redistributed surplus by deepening its own queue

File(s): allocate.py, cli.py

Description: allocate(...) places each new validator with the operator furthest below its target ratio, converging the active fleet toward the configured ratios. When capacity (an operator's pending-key count) is passed, an operator that cannot fill its share has its slots flow to operators that still have keys:

```
def has_room(o: Operator) -> bool:
    return capacity is None or alloc[o.name] < capacity.get(o.name, 0)
# ...
candidates = [o for o in live if has_room(o)]
```

So an operator that keeps a deep, mature pending queue while peers stay shallow repeatedly absorbs the redistributed surplus, gaining active-validator share (and the rewards that follow it) above its target ratio. The drift is recorded and printed, but only as a non-blocking warning in _print_preview(...); there is no concentration gate among the seven, so the deposit proceeds. It is self-correcting (an over-share operator carries a negative deficit until peers catch up), and min_key_age (12h) only rate-limits how fast a fresh queue can be built. No funds are at risk; absorbed validators still use the verified protocol/pod withdrawal credential.

Impact: Off-ratio concentration toward whichever operator keeps the deepest queue; economic and ratio drift only, no fund loss or wrong credentials.

Recommendation(s): Confirm with the protocol owner whether the concentration warning should become a blocking gate above some threshold rather than advisory auto-proceed.

Status: Acknowledged

Update from Swell: Confirming per the recommendation: the concentration warning stays advisory. We do not read a deep queue as steering — supplying mature keys is exactly what operators are asked to do, and surplus redistributes only when peers have no keys to take their own share. A blocking gate would leave user capital idle to penalize the one operator that came prepared, enforcing a ratio that self-corrects anyway (an over-share operator is skipped until peers catch up). Key maturity rate-limits how fast a queue can be deepened, and the post-run share is shown in the preview before confirm.

Update from CODESPECT: Acknowledged (accepted). The concentration signal stays an advisory warning, no code change.

8 Threat Model

Methodology

This threat model uses **actor-capability analysis** as its spine, cross-checked against an **invariant-violation matrix**. The actor lens is the right primary framing for the deposit bot because the tool is off-chain operator software that holds no protocol funds and has no privileged on-chain role of its own: its risk is dominated by what the parties it trusts (the registry that stores validator signatures, the RPC and beacon endpoints it reads, the operator who confirms and signs, and the local machine it runs on) can do if they are wrong, lagging, or hostile. We therefore enumerate, for each actor, what it may do when honest, what its blast radius is if compromised, and what bounds it, whether an in-bot gate or an on-chain contract backstop.

The invariant matrix is the rigour layer: each property the tool is expected to preserve (deposit under the correct withdrawal credential, no double deposit, funding sufficiency, key maturity, off-ratio drift surfaced rather than silent) is phrased as an attacker goal and matched against its mitigation. Where useful, threats are tagged with the informal STRIDE categories *Spoofing*, *Tampering*, *Elevation of privilege*, and *Denial of service*; *Repudiation* and *Information disclosure* are largely not applicable to a tool of this shape and are not enumerated separately.

Out of Scope

The Swell smart contracts the bot interacts with (`NodeOperatorRegistry`, `DepositManager`, `EigenLayerManager`, the exit contracts, and the `AccessControlManager`) are out of scope as software; the review treats them as trusted final backstops and models only the bot's interaction with them. The beacon deposit contract and the EigenLayer contracts are likewise out of scope. The correctness of the operator's key-generation and signature-submission process is out of scope as a process, but the *consequences* of a registry populated with wrong or malicious keys or signatures are modelled as a threat actor below. The security of the host machine (its operating system, the RPC/beacon endpoints it is pointed at, and the storage of the `DEPOSIT_BOT_KEY`) is an operational assumption, with the relevant compromise consequences modelled.

Assets

The following are what an attacker would seek to divert, corrupt, or deny, ranked roughly by impact:

a. **The 32 ETH per validator and its withdrawal-credential binding.** The whole purpose of a correct deposit is that the 32 ETH is recoverable via the intended withdrawal credential (the protocol `DepositManager` for swETH, the correct staker EigenPod for rswETH). A deposit made under a wrong or absent credential is the single most severe outcome, because it is the beacon chain, not the Swell contracts, that would accept it, and the stake is then effectively lost.

b. **Correct key selection and no double deposit.** Selecting the right pending keys, in registry-prefix order, exactly once. A repeat or out-of-order selection wastes funding or is rejected on-chain.

c. **The signing key.** `DEPOSIT_BOT_KEY` authorises the single deposit transaction; its confidentiality is an operational asset.

d. **Fleet ratio integrity.** That new validators are allocated towards the configured target ratios, and that any off-ratio drift or undeployed surplus is surfaced to the operator before confirmation rather than applied silently.

e. **The audit record.** The witness log's faithful, append-only account of every decision, which is what makes a run reconstructable after the fact.

Trust Boundaries

- **Bot ↔ registry (dominant boundary).** The bot selects keys but does not sign them; the deposit signatures it uses are registry-stored. The `bls_valid` gate is the boundary control: it turns "the registry returned these bytes" into "these bytes are a valid deposit signature committing to this exact withdrawal credential", which is also what recovers each rswETH key's pod.
- **Bot ↔ RPC / beacon.** The key-graph is read at the finalized block (reorg-safe and reproducible); money, obligation, and pause are read at latest to mirror the contract's live check; and the deposit root is read fresh immediately before sending. These anchors bound how much a lagging or adversarial endpoint can distort a plan before the contract backstops reject it.
- **Bot ↔ operator.** The operator supplies the signing key and gives the explicit go/no-go before the single irreversible broadcast; the preview it confirms against is, by construction, the same computation `execute` runs.
- **Bot ↔ local state.** The SQLite maturity index is a rebuildable cache of registry logs, not authoritative state: deleting it triggers a rebuild, and it is only ever advanced to a finalized block.

- **Bot ↔ chain (final authority).** The on-chain contract backstops (`InvalidDepositDataRoot`, `NextOperatorPubKeyMismatch`, `InsufficientETHBalance`) revert a bad plan regardless of what the bot computed, so a gate is an early, legible failure rather than the last line.

Threat Actors

Each actor is described by its honest capabilities, its blast radius if compromised, and the on-chain or in-bot bounds that constrain it.

Node operator (runs the bot)

Honest capabilities: configure a dataset, choose when to run, confirm or decline the broadcast, and supply the signing key.

If the host or key is compromised: an attacker with the signing key and the ability to run the tool can broadcast a deposit that the operator did not intend. The blast radius is bounded by what the gates and the contract backstops accept: keys must be the registry's pending prefix for real operators, the batch must be funded, no key may already be on the beacon chain, and, decisively, every key's registry signature must verify against its intended withdrawal credential. The attacker cannot mint a deposit under an arbitrary attacker-controlled credential unless the registry already holds a signature committing to it. *(Spoofing / Elevation of privilege.)*

Malicious or mistaken registry contents

Model: the registry is populated (by an operator, honestly or otherwise) with a key whose stored signature commits to a withdrawal credential other than the protocol's intended one, or with a freshly injected key placed at the deposit front of a shallow queue.

Threats and bounds:

- *wrong-credential deposit* — a key whose signature does not verify against the intended credential is blocked by `bls_valid`. For rswETH specifically, a key that verifies against no enumerated pod is unverified and blocks the run, so a deposit is never built under an unrecovered or attacker-substituted pod. *(Tampering.)*
- *just-added key funded before review* — guarded by `key_age_mature`, which refuses any key added less than `min_key_age` before the finalized block. A deep pending queue already keeps fresh keys away from the deposit front; the maturity gate is the guard for the shallow-queue case. A key missing an add-time is treated as age zero and fails closed. *(Tampering / Elevation of privilege.)*
- *out-of-order or duplicate keys* — bounded by front-of-queue prefix selection, the `no_duplicate_pubkeys` gate, and the on-chain `NextOperatorPubKeyMismatch` backstop. *(Tampering.)*

RPC / beacon endpoint (lagging or adversarial)

Model: the read RPC or beacon endpoint returns stale, partial, or manipulated data.

Threats and bounds:

- *stale key-graph* — the key-graph is pinned to the finalized block, so it cannot reorg and every pending key there has been seen by the maturity index; a coherent snapshot is used rather than a mix of block heights.
- *stale funding / pause* — read at latest to mirror the contract's own live check; the `funding_covers_queue` and `not_paused` gates fail early, and the contract re-checks both live (`InsufficientETHBalance`, plus the pause path).
- *stale deposit root / front-run* — the root is read fresh immediately before sending, and the beacon deposit contract reverts with `InvalidDepositDataRoot` if the passed root is not its current one, so a deposit built on a stale root never lands.
- *missing on-beacon data* — `not_on_beacon` depends on the beacon endpoint; an endpoint that omits a validator could let an already-deposited key pass this gate, but the registry's key consumption and the on-chain sequence backstop still prevent an actual double deposit. *(Tampering / Spoofing.)*
- *pruned or dishonest full-history RPC* — the maturity index requires full-history logs to build; a node that cannot serve them prevents the index from syncing, which fails the maturity gate closed (keys without an add-time are treated as age zero) rather than passing an unverified key. *(Denial of service.)*

Ethereum network / MEV

Model: reorgs and mempool observers around the deposit transaction.

Threats and bounds: the key-graph's finalized anchor removes reorg exposure for selection and maturity; the fresh-root guard removes the front-run window on the deposit root; and the broadcast path records the transaction hash before the single irreversible send, so an ambiguous in-flight failure is reported honestly and a re-run re-reads state without double-depositing. *(Denial of service.)*

Local maturity index (tampering)

Model: the SQLite cache is edited to backdate a key's add-time so a just-added key passes `key_age_mature`.

Bounds: the index is a rebuildable cache scoped per registry and only ever advanced to a finalized block; treating it as authoritative is an operational assumption, and a rebuild from registry logs restores ground truth. Tampering requires local write access, which is already the compromise boundary of the host actor above. *(Tampering.)*

Invariant / Violation Matrix

Invariant	How it could be violated	Mitigation
Every deposit binds the intended withdrawal credential (DepositManager for swETH; correct EigenPod for rswETH).	Registry signature commits to a different or attacker-chosen credential; wrong pod matched for rswETH.	`bls_valid` verifies each signature against the intended credential; rswETH pod is the unique one whose credential verifies, and a no-match key blocks the run.
No key is funded before it is mature enough to review.	A freshly injected key sits at the deposit front of a shallow queue.	`key_age_mature` against a finalized-block anchor; missing add-time fails closed.
Each key is deposited at most once, in registry-prefix order.	Duplicate or out-of-order selection; rerun after a partial send.	Front-of-queue prefix selection, `no_duplicate_pubkeys`, on-chain `NextOperatorPubKeyMismatch`, and registry key consumption on re-run.
A batch is fully funded.	Overcount validators versus available ETH net of exits.	`funding_covers_queue` mirrors the contract invariant; `InsufficientETHBalance` is the backstop.
A deposit is never built on a stale deposit root.	Root moves between plan and send (natural or front-run).	Root read fresh immediately before send; `InvalidDepositDataRoot` reverts a stale-root deposit.
Off-ratio drift and undeployed surplus are never silent.	Shallow queues redistribute or strand funded slots.	Dual (ratio-only vs capacity-aware) allocation surfaces redistributed and undeployed counts and post-run concentration in the preview and witness before confirm.
No deposit is sent without operator consent.	Automated or unattended broadcast.	Explicit confirm gate before the single irreversible send; `preview` never sends.

Table 4: Invariant / violation matrix for the deposit bot.

9 Evaluation of Provided Documentation

The deposit bot was documented in three complementary forms: a top-level `README.md`, two per-protocol design specifications under `docs/`, and thorough in-code module docstrings.

- **README.md:** A concise operator-facing overview. It states the tool's purpose (one run deposits one transaction; run again to deposit more), the `preview` / `execute` usage, and a seven-step account of what a run does. Valuably, it is explicit about what the tool does **not** do (it does not generate signatures and does not decide when to deposit), which pins down the trust boundary between the bot and the registry. It also maps the module layout and the develop/test workflow.
- **docs/DESIGN-sweth.md and docs/DESIGN-rsweth.md:** Two standalone design specifications, one per product, that together act as the behavioural specification for the tool. Each walks the full run end-to-end, states the finalized-versus-latest read split and the reasoning behind it, and enumerates the safety gates alongside the contract backstops that mirror them. The rswETH document additionally specifies the EigenPod recovery mechanism (a BLS signature commits to exactly one credential, so at most one pod can match), the maturity index and its full-history-RPC requirement, and the broadcast/audit model with its explicit treatment of the ambiguous post-broadcast failure. Keeping the two products in separate documents makes the single meaningful difference between them, the withdrawal credential, easy to isolate.
- **In-code docstrings:** Documentation is applied consistently at the module level and on the non-obvious functions. Each module opens with a docstring stating its role and its trust posture (for example, `bls.py` explains precisely why the SSZ signing-root construction is hand-rolled while the pairing verification is delegated to a maintained library, and `broadcast.py` documents the split of failure handling around the single irreversible send). The accounting-sensitive helpers (allocation shortfall, maturity-index latest-add-wins, the finalized/latest read buckets) carry inline rationale, not just descriptions of behaviour.

The documentation quality is high and, importantly, honest: it repeatedly states not only what the tool does but why each design choice was made and what it deliberately does not attempt. The clear articulation of the trust boundaries (bot to registry, bot to RPC/beacon, bot to operator) and of the gate-versus-backstop relationship gives the review a precise statement of intended behaviour to test against. **CODESPECT** recommends maintaining this standard as the tool evolves, in particular keeping the two `DESIGN` documents in sync with the gate set and the deposit-transaction shapes should new products or contract revisions be added.

10 Test Suite Evaluation

10.1 Build and Test Setup

The tool is a Python package (`requires-python >= 3.12`) managed with `uv`. Its runtime dependencies are minimal and deliberately confined: `web3` (used only in the chain and broadcast seams) and `blspy` (the blst-backed BLS binding, used only in `bls.py`). The trusted kernel imports the standard library only. Tests run with `pytest`.

```
uv sync --python 3.12
uv run pytest        # offline suite; the fork test is opt-in: pytest -m fork
```

10.2 Tests Output

```
platform linux -- Python 3.12.13, pytest-9.1.0, pluggy-1.6.0
rootdir: /tmp/eth-staking-deposit-bot
configfile: pyproject.toml
testpaths: tests
collected 67 items / 2 deselected / 65 selected

tests/test_allocate.py .................                                 [ 26%]
tests/test_bls.py .....                                                  [ 33%]
tests/test_broadcast.py .........                                        [ 47%]
tests/test_build_tx.py ..                                                [ 50%]
tests/test_full_pipeline.py .                                            [ 52%]
tests/test_kernel.py ...                                                 [ 56%]
tests/test_key_index.py ......                                           [ 66%]
tests/test_plan.py ..............                                        [ 87%]
tests/test_pod_recovery.py ..                                            [ 90%]
tests/test_sweth.py ..                                                   [ 93%]
tests/test_web3_chain.py ....                                            [100%]

============================================= warnings summary =============================================
.venv/lib/python3.12/site-packages/websockets/legacy/__init__.py:6
  /tmp/eth-staking-deposit-bot/.venv/lib/python3.12/site-packages/websockets/legacy/__init__.py:6: DeprecationWarning:
   ↪  websockets.legacy is deprecated; see https://websockets.readthedocs.io/en/stable/howto/upgrade.html for upgrade
   ↪  instructions
    warnings.warn(  # deprecated in 14.0 - 2024-11-09

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================ 65 passed, 2 deselected, 1 warning in 2.92s ================================
```

10.3 Notes on the Test Suite

The suite comprises 62 test functions across 12 files. All but one run fully offline; the single opt-in integration test (`test_fork_broadcast.py`, marked `fork`) executes a real rswETH deposit against a forked mainnet and is deselected by default. Offline tests exercise the code against a test double loaded with mainnet state recorded by `tools/record_fixture.py` (`tests/data/recorded-{sweth,rsweth}.json`), so the pure kernel is tested without a live node.

Coverage is well-targeted at the highest-stakes logic:

- **Allocation and planning** (`test_allocate.py`, `test_plan.py`): 27 tests over the ratio allocation, the capacity-bounded shortfall (redistributed and undeployed slots), key selection in registry order, and the end-to-end plan build.
- **BLS verification and pod recovery** (`test_bls.py`, `test_pod_recovery.py`, `test_build_tx.py`): the signing-root construction is validated against a real mainnet Swell validator, and the rswETH pod recovery and deposit-transaction encoding are checked against recorded pod sets.
- **Broadcast path** (`test_broadcast.py`): the irreversibility boundary, distinguishing pre-broadcast aborts (nothing sent) from the ambiguous post-broadcast case, and the operator-confirm and simulate-revert paths.
- **Maturity index** (`test_key_index.py`): the per-registry watermark, the latest-add-wins rule, and rebuild behaviour.
- **Chain seam and full pipeline** (`test_web3_chain.py`, `test_sweth.py`, `test_kernel.py`, `test_full_pipeline.py`): the read decoding and a preview-to-plan pipeline run over recorded data.

The suite is structured to match the code's kernel-and-seams design: the pure kernel is covered directly and deterministically from recorded fixtures, while the parts that must touch a live chain are isolated behind seams and exercised by the opt-in fork test.